

AOI 매칭 검증 앱 · 기술 분석 보고서

Camtek ↔ KLA 좌표변환 분석 보고서

서로 다른 두 AOI 장비가 같은 웨이퍼의 같은 결함을 서로 다른 숫자로 기록한다.
이 문서는 그 숫자들이 **어떤 파일에서 읽혀, 어떤 식으로 계산되어,**
하나의 좌표계 (col, row, x, y) 로 합쳐지는지를 코드 단위로 기술한다.

대상 코드	aoi_verification/app/coords/ · workers/coord_matcher.py
대상 제품	TB500 (die 7 × 6, 유효 die 30)
좌표 소스	ColorImageGrabingInfo.ini · KLA *.001 · LIVE 파일명
실측 데이터	dev/좌표 확인/ (Camtek 3폴더 · KLA 3폴더)
작성일	2026-07-28

이 문서의 모든 숫자는 실행으로 검증되었습니다

본문의 변환 결과·거리·축 방향은 저장소의 파서를 실제로 실행하고 dev/좌표 확인/ 의 실물 파일을 파싱해 얻은 값입니다. 추정으로 쓴 부분은 “**관례상**”이라고 명시했습니다.

§ 0 이 문서를 읽는 법

이 보고서는 두 종류의 독자를 함께 상대한다. “그래서 이게 무슨 뜻이냐”를 알고 싶은 사람과, “이 줄을 고치면 무엇이 깨지느냐”를 알아야 하는 사람이다. 앞부분 (§ 1- § 3)은 그림과 실제 숫자로, 뒷부분 (§ 4- § 9)은 코드와 파일 형식으로 같은 내용을 두 번 설명한다.

이런 목적이라면	이 순서로 읽으면 된다
개념만 빠르게 10분	§ 1 한 장 요약 → § 2 그림으로 이해하기 → § 7 실측 검증
매칭이 안 될 때 원인 추적	§ 3 참조 파일 형식 → § 8 매칭 엔진 → § 10 알려진 한계 → dev/diagnose_kla_camtek_coords.py
변환식·상수를 수정	§ 5 계산식 → § 6 좌표계 정합 → § 9 테스트 맵 (수정 후 반드시 회귀 테스트)
새 장비/제품 지원 추가	§ 5.5 상수표 → § 10 ①(TB500 하드코딩) → § 6 축 방향 분석

1부 — 개념 (그림과 예시)

- § 1 한 장 요약 — 무엇을, 왜, 어디서 계산하나
- § 2 그림으로 이해하기 — 웨이퍼 격자, 두 장비의 번지수, 예시 1건 완주
- § 3 참조 파일 3종의 실물 형식 — INI · KLA .001 · LIVE 파일명

2부 — 코드 (개발자용)

- § 4 데이터 모델 DefectCoord
- § 5 소스별 계산식과 파싱 구조 — 우선순위 체인, 3개 파서, 상수표
- § 6 좌표계 정합 — 왜 row 는 Camtek 만, y 는 KLA 만 뒤집는가
- § 7 실측 검증 — 샘플 9장의 실행 결과
- § 8 매칭 엔진 — ±1 게이트 · 거리 · score 인코딩
- § 9 결과 표기와 테스트 맵

3부 — 유지보수

- § 10 알려진 한계와 주의사항 — 7건
- § A 부록 A. 용어집
- § B 부록 B. 파일·함수 인덱스

§ 1 한 장 요약

1.1 무엇을 계산하는가

같은 웨이퍼를 Camtek 장비와 KLA 장비가 각각 검사하면, **같은 결함**이 두 장비의 결과 폴더에 서로 다른 이름·다른 숫자로 남는다. 좌표 매칭 모드는 이미지를 비교하지 않고 **결함의 물리적 위치**를 비교하므로, 두 장비의 좌표를 먼저 같은 언어로 번역해야 한다. 그 번역 결과가 네 개의 숫자다.

`col, row` — 웨이퍼 위 몇 번째 칸(die)인가 (정수, 0-based)
`x, y` — 그 칸 안에서 어디인가 (μm, die 내부 local 좌표 — 절대 웨이퍼 좌표가 아니다)

이 분리가 설계의 핵심이다. **(col, row)**는 **이산 키**로 후보를 거르는 데만 쓰고, **(x, y)**는 **연속값**으로 die 안에서의 거리만 잰다. 두 값을 섞어 하나의 절대 좌표로 합치는 코드는 어디에도 없다.

1.2 어디서 계산하는가

이미지 경로 (Path)

```

▼
coords.resolve()                                coords/__init__.py:20
├─ 1순위 camtek_live.resolve()  파일명 정규식만 사용 (외부 파일 불필요)
├─ 2순위 camtek_ini.resolve()   같은 폴더의 ColorImageGrabingInfo.ini
└─ 3순위 kla_info.resolve()     같은 폴더의 *.001 (KLA 결과 파일)
▼
DefectCoord(col, row, x, y, source, native_x, native_y)
├─ workers/coord_matcher.py  매칭 : (col,row) ±1 게이트 → (x,y) 유클리드 거리 → score
└─ workers/exporter.py:517   엑셀 : "col 5 / row 3" · "x 11819 / y 31035 μm"

```

좌표를 만드는 코드는 `aoi_verification/app/coords/` 패키지 **한 곳뿐**이고, 계산 자체는 **상수 6개와 사칙연산**이 전부다. 무거운 의존성(PyQt6·cv2·OpenVINO)이 전혀 없어 헤더리스로 단위 테스트된다.

1.3 결론 한 표

두 장비의 원본 필드가 표준 좌표로 변환되는 전체 대응

표준	Camtek ColorImageGrabingInfo.ini	KLA *.001
col	Col - 2	XINDEX + 3
row	6 - Row ← 반전	YINDEX + 3
x	X - Col × 37247.7	round(XREL)
y	Y - Row × 44905.4	round(DiePitchY - YREL) ← 반전

가장 자주 오해되는 지점

인덱스 반전은 Camtek 쪽에만 (6 - Row), **die 내부 좌표 반전은 KLA 쪽에만** (DiePitchY - YREL) 걸려 있다. 실수처럼 보이지만 **둘 다 있어야** 두 장비가 일치한다 — 이유는 § 6에서 실측으로 증명한다.

§ 2 그림으로 이해하기

이 절은 코드를 몰라도 읽을 수 있게 썼다. 웨이퍼를 바둑판, die 를 칸, 결함을 칸 안에 찍힌 점이라고 생각하면 된다. 두 장비는 같은 바둑판을 쓰지만 칸에 번호를 매기는 방식과 칸 안에서 자를 대는 방향이 다르다.

2.1 TB500 웨이퍼의 실제 모양

TB500 은 가로 7칸 × 세로 6칸의 격자지만, 웨이퍼가 둥글기 때문에 네 모서리가 잘려 실제로 검사되는 die 는 30개다. 아래 표는 KLA 결과 파일의 SampleTestPlan 항목(30줄)을 그대로 옮겨 그린 것이다.

표준 col →	0	1	2	3	4	5	6
Camtek Col →	2	3	4	5	6	7	8
KLA XINDEX →	-3	-2	-1	0	1	2	3
row 5 Row 1 YI 2			2,5	3,5	4,5 예시 2		
row 4 Row 2 YI 1		1,4	2,4	3,4	4,4	5,4 예시 3	
row 3 Row 3 YI 0	0,3	1,3	2,3	3,3	4,3	5,3 예시 1	6,3
row 2 Row 4 YI -1	0,2	1,2	2,2	3,2	4,2	5,2	6,2
row 1 Row 5 YI -2		1,1	2,1	3,1	4,1	5,1	
row 0 Row 6 YI -3			2,0	3,0	4,0		

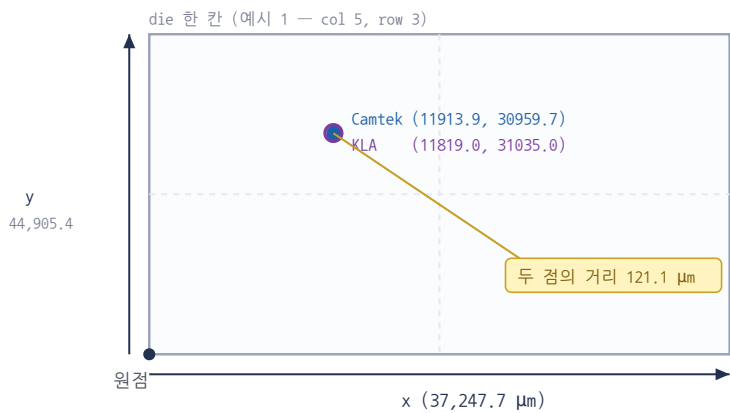
▽ notch (KLA 헤더 OrientationMarkLocation DOWN)

☐ 검사 대상 die (30) ☐ 웨이퍼 밖 (12) ☒ 본문 예시 위치

같은 칸에 세 개의 번지수가 붙어 있는 것이 이 표의 요점이다. Camtek 은 Col 2..8 / Row 1..6 , KLA 는 XINDEX -3..3 / YINDEX -3..2 , 그리고 애플이 쓰는 표준은 col 0..6 / row 0..5 다. 세로 방향만 Camtek 과 KLA 가 서로 반대로 번호를 매기는 것도 표에서 바로 보인다 (Row 1 ↔ YI 2 , Row 6 ↔ YI -3).

2.2 칸 안에서의 위치 — x, y

칸 하나의 크기는 가로 37,247.7 μm × 세로 44,905.4 μm(약 37 mm × 45 mm)다. x · y 는 그 칸의 한 귀퉁이를 원점으로 켜 거리다. 두 장비가 같은 결함을 찍었다면 이 값이 거의 같아야 한다 — 완전히 같지는 않다 (장비 측정 오차 + 반올림).



그림의 두 점이 **121 μm** 떨어져 있다. 기본 허용 오차 500 μm 안이므로 “같은 결함”으로 판정된다.

2.3 예시 1건 완주 — 숫자만 따라가기

같은 결함을 두 장비가 어떻게 기록했고, 각각 무엇을 계산해 같은 답에 도달하는지 끝까지 따라가 보자. 아래 값은 `dev/좌표` 확인/ 의 실제 파일에서 그대로 가져온 것이다.

Camtek — 예시 1

272646.165679.c.1000203959.2.jpeg

원본 X 272647.761

원본 Y 165675.854

원본 Col 7

원본 Row 3

col $7 - 2 = 5$

row $6 - 3 = 3$

x $272647.761 - 7 \times 37247.7$
= 11913.861

y $165675.854 - 3 \times 44905.4$
= 30959.654

KLA — 예시 1

W6459076XYG1_2_0_23_2.jpg

XREL 11819.421

YREL 13870.779

XINDEX 2

YINDEX 0

DiePitchY 44905.34

col $2 + 3 = 5$

row $0 + 3 = 3$

x $\text{round}(11819.421)$
= 11819

y $\text{round}(44905.34 - 13870.779)$
= 31035

판정

col 차이 $|5 - 5| = 0 \leq 1 \checkmark$ · row 차이 $|3 - 3| = 0 \leq 1 \checkmark$

거리 $\sqrt{(11913.861 - 11819)^2 + (30959.654 - 31035)^2} = 121.1 \mu\text{m} < 500 \mu\text{m} \checkmark \rightarrow$ 같은 결함으로 매칭 성공

전혀 다르게 생긴 두 묶음의 숫자가 같은 (5, 3) 과 거의 같은 ($\approx 11.9 \text{ mm}$, $\approx 31.0 \text{ mm}$) 로 수렴한다. 이 수렴을 만들어내는 것이 § 5 의 변환식이고, 수렴이 왜 성립하는지가 § 6 이다.

§ 3 참조 파일 3종의 실물 형식

좌표는 이미지 파일 안에 들어 있지 않다. 이미지와 같은 폴더에 있는 메타데이터 파일, 또는 **파일명 자체**에서 읽는다. 세 가지 소스의 실물 형식을 그대로 보인다.

3.1 Camtek ColorImageGrabingInfo.ini

Windows INI 형식이고, **섹션 이름이 곧 이미지 파일명**이다. 한 폴더의 모든 결함 이미지가 한 파일에 모여 있다.

```
dev/좌표 확인/Camtek/예시1/ColorImageGrabingInfo.ini (실물 발췌)

[272646.165679.c.1000203959.2.jpeg] ← 섹션명 = 이미지 파일명
X=272647.761085349 ← 절대 wafer X (μm) [필수]
Y=165675.853620774 ← 절대 wafer Y (μm) [필수]
StageZ=74546.8
FaultX=272647.761085349 ← X 가 없을 때의 폴백
FaultY=165675.853620774 ← Y 가 없을 때의 폴백
PixelSizeX=0.172499999403954
PixelSizeY=0.172499999403954
Mag=20
OpticName=x20
RecipeNumber=1
ImageSizeX=1380
ImageSizeY=1036
Col=7 ← die column (1-based) [필수]
Row=3 ← die row (1-based) [필수]
Type=0
Index=13
IsTableCoordinate=0
IsColor=1
... (Is* 플래그 20여 개 — 좌표와 무관)

[105563.142613.c.1886791976.1.jpeg] ← 다음 이미지 섹션이 이어짐
...
```

좌표 계산에 쓰이는 키 (나머지 키는 전부 무시)

키	예시 값	의미와 쓰임
X	272647.761	절대 wafer X(μm). x 계산의 피감수. 없으면 FaultX 를 쓴다.
Y	165675.854	절대 wafer Y(μm). y 계산의 피감수. 없으면 FaultY .
Col	7	die column. col 계산과 x 의 die 원점 계산에 모두 쓰인다.
Row	3	die row. row 계산과 y 의 die 원점 계산에 모두 쓰인다.

네 키 중 하나라도 없으면 그 이미지는 좌표 없음

`_extract_coord` 는 `if None in (X, Y, Col, Row): return None` 으로 즉시 포기한다(`camtek_ini.py:86`). 부분 복구는 하지 않는다.

3.2 KLA *.001 결과 파일

KLA 표준 결과 파일(KLARF 계열)이다. 헤더에 웨이퍼 전체 정보가 있고, 그 아래로 TiffFileName (이미지 파일명) 과 DefectList (결함 수치) 가 번갈아 반복된다.

dev/좌표 확인/KLA/예시1/00cd0487-..._TB500INT.271@6324_W6459076XYG1.001 (실물 발체)

```

FileVersion 1 2;
FileTimestamp 05-19-26 23:04:28;
InspectionStationID "KLA" "LDS CIRCL" "K3";
SampleType WAFER;
LotID "TB500INT.271@6324";
DeviceID "4DT-TB500-H-M1";
StepID "RDL2_NEW(MULTI)";
SampleOrientationMarkType NOTCH;
OrientationMarkLocation DOWN;           ← notch 위치
DiePitch 3.7247930000e+004 4.4905340000e+004;   ← DiePitchX, DiePitchY (μm)
DieOrigin 0.0000000000e+000 0.0000000000e+000;
WaferID "W6459076XYG1";                 ← 슬롯명(WaferID) — 헤더에 명시
Slot 10;
SampleCenterLocation 1.8460000000e+004 -2.2294100000e+002;
ClassLookup 12
  0 "Unclassified"   3 "Metal Residue"   1 "Undersized Bump"
  ...
                                (결함 분류 코드표)
SampleTestPlan 30
-3 -1  -3  0  -2 -2  -2 -1  -2  0  -2  1
-1 -3  -1 -2  -1 -1  -1  0  -1  1  -1  2
  0 -3   0 -2   0 -1   0  0   0  1   0  2
  1 -3   1 -2   1 -1   1  0   1  1   1  2
  2 -2   2 -1   2  0   2  1   3 -1   3  0 ;   ← § 2.1 웨이퍼 맵의 출처
DefectRecordSpec 22 DEFECTID X Y XREL YREL XINDEX YINDEX XSIZE YSIZE
                   DEFECTAREA DSIZE CLASSNUMBER TEST ... IMAGELIST ;
DefectList;
TiffFileName W6459076XYG1_2_0_23_1.jpg;      ← 이미지 ①
DefectList
  1 70567.720 37204.350 14531.861 36981.409 2 0 1.300 1.300 1.690000 ... ;
TiffFileName W6459076XYG1_2_0_23_2.jpg;      ← 이미지 ② (예시 1)
DefectList
  2 67855.280 14093.720 11819.421 13870.779 2 0 2.600 3.900 10.160000 ... ;
TiffFileName W6459076XYG1_1_-2_7_3.jpg;
DefectList
  3 41850.190 -51480.578 23062.260 38107.159 1 -2 1.300 1.300 1.690000 ... ;

```

DefectList 행의 앞 7개 필드 (0-based 위치 — 코드가 고정 가정하는 부분)

#	필드	예시 ② 값	의미
0	DEFECTID	2	결함 일련번호
1	X	67855.280	절대 wafer X (μm) — 좌표 변환에는 쓰지 않음
2	Y	14093.720	절대 wafer Y (μm) — 좌표 변환에는 쓰지 않음
3	XREL	11819.421	die 내부 상대 X (μm) → <code>x</code>
4	YREL	13870.779	die 내부 상대 Y (μm) → <code>y</code> (반전 후)
5	XINDEX	2	die X 인덱스 (중심 0, 부호 있음) → <code>col</code>
6	YINDEX	0	die Y 인덱스 (중심 0, 부호 있음) → <code>row</code>

KLA 이미지 파일명의 구조

```
W6459076XYG1 _ 2 _ 0 _ 23 _ 2 .jpg
└─ WaferID ─┬─┬─XI─┬─YI─┬─ID─┬─T─┘
```

파일명에도 XINDEX·YINDEX 가 들어 있어 파일명만으로 col/row 를 뽑을 수 있어 보이지만, **코드는 파일명을 좌표 계산에 전혀 쓰지 않는다**. 파일명은 `.001` 안의 `TiffFileName` 과 대조하는 **키로만** 쓰이고, 수치의 유일한 출처는 항상 `.001` 이다.

3.3 Camtek LIVE 파일명

LIVE 모드로 저장된 Camtek 이미지는 **파일명 자체에 좌표가 인코딩**되어 있어 별도 메타데이터 파일이 필요 없다. 두 가지 배치가 존재한다.

형식 A — DefectName 이 x/y 뒤 (또는 없음)

```
R_TB500_LIVE_P14_PXU-PIDS3_00RMF043XYE0_5_3_21620.211_7230.807_Foreign Material
                                col row      x              y      DefectName
```

형식 B — DefectName 이 x/y 앞

```
R_TB500_LIVE_P14_VLP-PDIS3_W6317098XYB5_4_5_Over Sized Bump_30229.803_1987.994
                                col row      DefectName      x              y
```

LIVE 파일명은 col 과 row 의 취급이 다르다

`col` 은 그대로 쓰고, `row` 는 **-1** 한다 (`camtek_live.py:39`). 파일명의 row 토큰이 Camtek 표시 규약(1..6)이라는 전제 때문이다. `x·y` 는 이미 die 내부 μm 값이라 변환하지 않는다.

§ 4 데이터 모델 DefectCoord

세 소스의 결과는 전부 이 한 클래스로 수렴한다. 이후 매칭·엑셀 출력은 원본 파일이 무엇이었던지 알 필요가 없다.

aoi_verification/app/coords/models.py:14

```
@dataclass(frozen=True)
class DefectCoord:
    """변환 완료된 defect 좌표 — 세 소스(Camtek INI / LIVE 파일명 / KLA .001) 공통 표현."""
    col: int      # 0-based 표준 die column (Camtek/KLA 통일)
    row: int      # 0-based 표준 die row
    x: float      # die 내부 local X (μm)
    y: float      # die 내부 local Y (μm)
    source: str   # "camtek_ini" | "camtek_live" | "kla"
    # KLA 원본 die-내부 좌표(XREL/YREL, μm) — source="kla" 일 때만 채운다.
    # 엑셀에 'Camtek 좌표계 변환값'과 'KLA 자체 좌표값'을 함께 표기하기 위한.
    native_x: Optional[float] = None
    native_y: Optional[float] = None
```

필드	단위	설명 / 주의
col	정수	0-based. TB500 에서 0..6.
row	정수	0-based. TB500 에서 0..5. Camtek 쪽만 반전을 거친 값 (§ 6).
x	μm	die 내부 좌표. 절대 웨이퍼 좌표가 아니다. KLA 는 정수로 반올림된다.
y	μm	die 내부 좌표. KLA 쪽만 축 반전을 거친 값 (§ 6).
source	문자열	어느 파서가 만들었는지. 엑셀에서 KLA 원본 병기 여부를 가른다.
native_x/y	μm	KLA 원본 XREL/YREL. 변환 전 값을 잃지 않기 위한 보존 필드 . Camtek 소스에서는 항상 <code>None</code> .

절대 좌표가 필요한 경로는 완전히 별개다

결함 형상(area/width/length)을 `Surface.flt` 에서 조회할 때는 절대 좌표가 필요하다. 이때는 `DefectCoord` 를 쓰지 않고 `coords/abs_coord.py` 가 INI 의 원시 `x / y` (또는 점표기 파일명의 앞 두 토큰)를 **가공 없이** 돌려준다. 좌표 매칭 경로와 형상 조회 경로는 서로 간섭하지 않는다.

§ 5 소스별 계산식과 파싱 구조

5.1 우선순위 체인

`coords.resolve()` 는 세 파서를 **순서대로** 시도하고 첫 성공을 반환한다. 비용이 싼 것부터 시도하는 구조다 — LIVE 는 파일 I/O 가 아예 없고, INI·KLA 는 폴더 단위 캐시가 걸린다.

aoi_verification/app/coords/_init_.py:20

```
def resolve(image_path: Path) -> Optional[DefectCoord]:
    """이미지 경로 → DefectCoord. 세 소스를 순서대로 시도, 모두 실패하면 None."""
    coord = camtek_live.resolve(image_path) # 1순위 — 파일명 정규식
    if coord is not None:
        return coord
    coord = camtek_ini.resolve(image_path) # 2순위 — ColorImageGrabingInfo.ini
    if coord is not None:
        return coord
    return kla_info.resolve(image_path) # 3순위 — *.001
```

`resolve_batch(paths)` 는 이 함수를 단순 반복하지만, INI/KLA 파서에 걸린 `@lru_cache(maxsize=256)` 덕분에 **같은 폴더의 파일은 한 번만 읽는다**. 매칭 시작 시 전체 이미지를 한 번에 프리패치하는 것도 이 때문이다 (`coord_matcher.py:162`).

5.2 Camtek INI 파서

파싱

`configparser` 를 쓰지 않는다. 섹션명에 `.` 과 특수문자가 섞인 파일명이 그대로 들어가 표준 파서와 잘 맞지 않기 때문이다. 대신 정규식으로 통째로 split 한다.

aoi_verification/app/coords/camtek_ini.py:52

```
def _parse_ini(path: Path) -> dict[str, DefectCoord]:
    text = path.read_text(encoding="utf-8", errors="replace")

    # 섹션 단위로 분리: [filename.jpeg] → 내용 반복
    parts = _SECTION_PAT.split(text)
    # parts[0] = 섹션 전 텍스트(무시), 이후 [이름, 내용, 이름, 내용, ...] 교대
    result: dict[str, DefectCoord] = {}
    it = iter(parts[1:])
    for name, content in zip(it, it): # zip(it, it) = 2개씩 묶기
        stem = Path(name.strip()).stem # "foo.jpeg" → "foo"
        coord = _extract_coord(content)
        if coord is not None:
            result[stem.lower()] = coord # 키는 항상 소문자 stem
    return result
```

파일 탐색은 대소문자 후보 두 개를 시도한다 — (`"ColorImageGrabingInfo.ini"`, `"ColorImageGrabinginfo.ini"`) (`camtek_ini.py:23`). 장비 버전에 따라 표기가 갈리기 때문이다.

계산

aoi_verification/app/coords/camtek_ini.py:89

```
col_i = int(Col) # 원본 Col — 정규화 전 값
row_i = int(Row) # 원본 Row — 정규화 전 값
col = col_i - CAMTEK_COL_OFFSET # = Col - 2
row = CAMTEK_ROW_TOTAL - row_i # = 6 - Row ← 인덱스 반전
x = X - col_i * CAMTEK_PITCH_X # = X - Col × 37247.7
y = Y - row_i * CAMTEK_PITCH_Y # = Y - Row × 44905.4
return DefectCoord(col=col, row=row, x=x, y=y, source="camtek_ini")
```

놓치기 쉬운 점 — x/y 는 표준화 전의 Col/Row 를 쓴다

x 계산의 곱셈 인자는 col (=Col-2)이 아니라 col_i (=Col)다. x 는 “절대 좌표 — 그 die 의 원점”이라는 독립적인 뽀렘이고, die 원점은 장비 원본 인덱스로 매겨져 있기 때문이다. 인덱스 표준화(-2 , 6-)와 die 내부 좌표 계산은 서로 다른 목적의 연산이다. 여기서 col / row 를 잘못 넣으면 x 가 한 칸(37 mm)씩 밀린다.

5.3 Camtek LIVE 파일명 파서

aoi_verification/app/coords/camtek_live.py:26

```
# 형식 A: ..._col_row_x[.xx]_y[.yy][_DefectName]
_PAT_A = re.compile(
    r'_(\d+)_(\d+)_([\d+](?:\.\d+)?)([\d+](?:\.\d+)?)(?:_.+)?$'
)
# 형식 B: ..._col_row_DefectName_x[.xx]_y[.yy] (DefectName 이 x/y 앞)
_PAT_B = re.compile(
    r'_(\d+)_(\d+)_+([\d+](?:\.\d+)?)([\d+](?:\.\d+)?)$'
)

def _extract(m) -> Optional[DefectCoord]:
    col = int(m.group(1))
    row = int(m.group(2)) - 1 # 표시 규약(1..6) → 0-based 표준(0..5)
    x = float(m.group(3))
    y = float(m.group(4))
    return DefectCoord(col=col, row=row, x=x, y=y, source="camtek_live")
```

KLA 파일명 오매칭 방지 — 이 파서의 진짜 어려움

KLA 이미지 파일명 W6459076XYG1_2_0_23_2.jpg 도 형식 A 정규식에 걸린다. 그대로 두면 KLA 이미지가 “col 2, row -1, x 23, y 2” 라는 완전히 엉뚱한 Camtek 좌표로 해석되고, 3순위 KLA 파서까지 가지도 못한다. 이를 막는 것이 한 줄의 접두부 검사다.

aoi_verification/app/coords/camtek_live.py:47

```
def resolve(image_path: Path) -> Optional[DefectCoord]:
    stem = image_path.stem
    for pat in (_PAT_A, _PAT_B):
        m = pat.search(stem)
        if m and '_' in stem[:m.start()]: ← 매칭 지점 앞에 '_' 가 있어야 함
            return _extract(m)
    return None
```

접두부 검사의 판별 원리

종류	파일명 stem	매칭 앞부분	결과
LIVE	R_TB500_LIVE_..._5_3_21620.211_7230.807	R_TB500_LIVE_...	☐ 있음 → 통과
KLA	W6459076XYG1_2_0_23_2	W6459076XYG1	☐ 없음 → 거부 → 3순위로
Camtek INI	272649.165672.c.1796377335.1	—	정규식 자체가 불일치 → 거부 → 2순위로

5.4 KLA .001 파서

정보 파일 탐색

aoi_verification/app/coords/kla_info.py:53

```
def _find_info_file(folder: Path) -> Optional[Path]:
    """폴더에서 KLA .001 정보 파일 탐색."""
    # 1순위: .001 확장자
    for p in folder.glob("*.001"):
        return p
    # 2순위: .jpg/.pass 가 아닌 파일 (확장자 없는 파일 포함), .pass 제외
    candidates = [
        p for p in folder.iterdir()
        if p.is_file()
        and p.suffix.lower() not in _NON_INFO_SUFFIXES # .jpg .jpeg .pass .ini .py
        and not p.name.startswith('.')
    ]
    if len(candidates) == 1:      ← 후보가 정확히 1개일 때만 채택
        return candidates[0]
    return None
```

KLA 결과 폴더의 정보 파일은 확장자가 **.001** 이 아니라 **아예 없을 수도** 있다. 그래서 2순위 규칙이 존재하는데, 후보가 2개 이상이면 어느 쪽이 정보 파일인지 확신할 수 없어 `None` 을 돌려준다 — 이 보수성이 § 10 ④의 한계로 이어진다.

파싱 — TiffFileName 과 DefectList 를 짚짓는 상태 기계

aoi_verification/app/coords/kla_info.py:141

```
def _parse_info_raw(path: Path) -> dict[str, dict]:
    text = path.read_text(encoding="utf-8", errors="replace")
    lines = text.splitlines()

    # ① DiePitchY 추출 — 파일 전체에서 한 번만
    die_pitch_y: Optional[float] = None
    for line in lines:
        m = _DIEPITCH_PAT.search(line)
        if m:
            die_pitch_y = float(m.group(2)) # DiePitch X Y → 두 번째 값
            break
    if die_pitch_y is None:
        return {} # DiePitch 없으면 전부 포기

    # ② TiffFileName → DefectList 를 번갈아 읽는 상태 기계
    result: dict[str, dict] = {}
    current_stem: Optional[str] = None
    for line in lines:
        tm = _TIFF_PAT.match(line)
        if tm: # TiffFileName 줄 → 현재 이미지 갱신
            current_stem = Path(tm.group(1).strip()).stem.lower()
            continue
        if current_stem is not None: # 그 뒤 첫 숫자 데이터 행 1개만
            dm = _DEFECT_PAT.match(line)
            if dm:
                result[current_stem] = {
                    "xrel": float(dm.group(4)), "yrel": float(dm.group(5)),
                    "xindex": int(dm.group(6)), "yindex": int(dm.group(7)),
                    "die_pitch_y": die_pitch_y,
                }
            current_stem = None # TiffFileName 당 DefectList 행 1개
    return result
```

필드 위치는 정규식 `_DEFECT_PAT` (`kla_info.py:31`)이 **0~6 위치로 고정** 가정한다. 파일에 `DefectRecordSpec` 으로 스키마가 명시돼 있지만 **읽지 않는다**(§ 10 ③).

계산

aoi_verification/app/coords/kla_info.py:128

```
@lru_cache(maxsize=256)
def load_folder(folder: Path) -> dict[str, DefectCoord]:
    result: dict[str, DefectCoord] = {}
    for stem, r in load_folder_raw(folder).items():
        col = r["xindex"] + KLA_ZERO_X # = XINDEX + 3
        row = r["yindex"] + KLA_ZERO_Y # = YINDEX + 3
        x = round(r["xrel"]) # 정수 μm 로 반올림
        y = round(r["die_pitch_y"] - r["yrel"]) # ← 축 반전 후 반올림
        result[stem] = DefectCoord(
            col=col, row=row, x=float(x), y=float(y), source="kla",
            native_x=float(r["xrel"]), native_y=float(r["yrel"]),
        )
    return result
```

파싱과 변환이 분리돼 있다

load_folder_raw() 는 원본값만, load_folder() 는 변환값만 담당한다. 진단 도구 dev/diagnose_kla_camtek_coords.py 가 “변환 전 값”을 볼 수 있도록 일부러 나눈 구조다. 변환식을 고칠 때는 load_folder() 만 손대면 된다.

덤: 슬롯명(WaferID) 판독은 좌표와 다른 규칙을 쓴다

같은 파일에서 WaferID "XXXX"; 를 읽는 read_wafer_id() (kla_info.py:88)는 좌표와 달리 후보 파일을 전부 훑고, 헤더만 필요하므로 앞 64 KB만 읽는다. 사진이 한 장도 없는 폴더도 식별되어야 하기 때문이다. 같은 폴더에서 슬롯명은 읽히는데 좌표는 못 읽는 상태가 가능한 이유가 이 비대칭이다.

5.5 상수표와 그 근거

```
aoi_verification/app/coords/models.py:44

# ----- TB500 Camtek INI 변환 상수 -----
CAMTEK_PITCH_X: float = 37247.7 # μm/die (TB500)
CAMTEK_PITCH_Y: float = 44905.4 # μm/die (TB500)
CAMTEK_COL_OFFSET: int = 2 # TB500
# TB500: Y die 수 6, INI 원본 Row 는 1-based 1..6 (dev/좌표 확인 실측 분포).
# 1-based 위→아래를 0-based 아래→위(row 0..5)로 뒤집는 값 - 7 이면 전 구간 +1 로 어긋난다
# (현물 웨이퍼 맵 · KLA 변환과 1 차이 나던 버그의 원인).
CAMTEK_ROW_TOTAL: int = 6 # TB500

# ----- TB500 KLA .001 변환 상수 -----
KLA_ZERO_X: int = 3 # TB500: package X count 7, 7 // 2 = 3
KLA_ZERO_Y: int = 3 # TB500: package Y count 6, 6 // 2 = 3
```

상수	값	쓰이는 곳	근거
CAMTEK_PITCH_X	37247.7	Camtek x	TB500 die 폭. KLA 헤더 값(37247.93)과 0.23 μm 차 → § 10 ②
CAMTEK_PITCH_Y	44905.4	Camtek y	TB500 die 높이. KLA 헤더 값(44905.34)과 0.06 μm 차
CAMTEK_COL_OFFSET	2	Camtek col	실측 Col 분포 2..8 → 0..6
CAMTEK_ROW_TOTAL	6	Camtek row	실측으로 교정된 값. 과거 7 이던 시절 전 구간 row 가 +1 어긋났다
KLA_ZERO_X	3	KLA col	패키지 X die 수 7 ÷ 2 (중심 0 → 0-based)
KLA_ZERO_Y	3	KLA row	패키지 Y die 수 6 ÷ 2

이 여섯 값은 전부 TB500 전용 하드코딩이다

다른 제품(다른 die pitch·die 수)에 그대로 쓰면 col/row/x/y 가 전부 틀리는데, 값이 그럴듯해서 예외 없이 조용히 오매칭된다. 제품 판별이나 방어 로직은 현재 없다.

§ 6 좌표계 정합 — 왜 한쪽만 뒤집는가

변환식을 처음 보면 **Camtek** 은 row 만 뒤집고 y 는 안 뒤집는데, **KLA** 는 row 는 안 뒤집고 y 만 뒤집는 비대칭이 오류처럼 보인다. 이 절은 그것이 왜 정확히 옳은지를 실측으로 보인다.

6.1 KLA 절대 좌표의 재구성 — XREL/YREL 의 방향 확정

.001 파일에는 절대 좌표 X / Y 와 상대 좌표 $XREL / YREL$ 이 둘 다 들어 있다. 이 둘의 관계를 풀면 $XREL / YREL$ 이 die 의 어느 모서리에서 어느 방향으로 재는 값인지 확인할 수 있다. 샘플 3개 폴더의 모든 결함 행에 대해 다음 식이 성립했다.

$$X = XINDEX \times DiePitchX + XREL - SampleCenterX$$
$$Y = YINDEX \times DiePitchY + YREL - SampleCenterY$$

(간차 $\leq 0.003 \mu m$ — 부동소수 오차 수준)

즉 **XREL** 과 **YREL** 은 둘 다 **KLA** 자체 측과 같은 방향으로, die 의 “낮은 쪽 모서리”에서 재는 값이다. **YREL** 만 방향이 다를 이유가 없다. 그렇다면 $y = DiePitchY - YREL$ 의 뺄셈은 “원점을 옮기는” 것이 아니라 “측을 뒤집는” 것이다.

기존 문서의 설명 정정

이전 판 보고서는 이 뺄셈을 “KLA 의 Y축 원점이 die 상단이므로 하단 기준으로 변환”이라고 설명했다. 위 재구성 결과와 맞지 않는다. 실제 이유는 두 장비의 Y축이 서로 반대 방향이고, 그 반대를 Camtek 좌표계 쪽으로 맞추기 위한 반전이다.

6.2 die 인덱스의 방향 — 실측 대응 관계

샘플 3쌍(같은 물리적 결함)의 원본 인덱스를 나란히 놓으면 대응이 바로 드러난다.

예시	Camtek Col	KLA XINDEX	관계	Camtek Row	KLA YINDEX	관계
1	7	2	$Col = XI + 5$	3	0	$Row = 3 - YI$
2	6	1	✓	1	2	✓
3	7	2	✓	2	1	✓

가로는 **평행 이동만** ($Col = XINDEX + 5$) 이고, 세로는 **부호가 뒤집힌다** ($Row = 3 - YINDEX$). 즉 두 장비는 X 방향은 같은 쪽으로, Y 방향은 반대쪽으로 번호를 매긴다. § 2.1 웨이퍼 맵의 세로축 라벨이 **Row 1** ↔ **YI 2** 로 엇갈리는 것이 이것이다.

6.3 그래서 반전이 두 번 필요하다

Camtek 쪽에서 하는 일

인덱스는 뒤집는다 — $row = 6 - Row$
die 내부 y 는 그대로 — $y = Y - Row \times pitch$

Camtek 의 Y 축 방향을 표준으로 삼았으므로 die 내부 좌표는 손덜 필요가 없다. 인덱스만 표준 방향(= KLA 방향)으로 뒤집는다.

KLA 쪽에서 하는 일

인덱스는 그대로 — $row = YINDEX + 3$
die 내부 y 는 뒤집는다 — $y = pitchY - YREL$

인덱스 방향은 이미 표준이다. 대신 die 내부 좌표가 Camtek 과 반대 방향이므로 $pitchY$ 에서 빼서 뒤집는다.

두 반전은 **서로 다른 층위**(칸 번호 vs 칸 내부 위치)에 걸쳐 있고, 각각 한 번씩만 적용된다. 어느 한쪽만 지우면 row 가 통째로 어긋나거나(인덱스) die 내부 y 가 위아래로 뒤집힌다.

실측으로 확인된 것과, 관례로 추정된 것의 구분

실측으로 확정: 표준 row 가 증가하는 물리 방향과, die 내부 표준 y 가 증가하는 방향은 **서로 반대**다. (두 장비 모두 동일하게 그렇다.)

관례로 추정: KLA 헤더가 OrientationMarkLocation DOWN (notch 아래)이고 웨이퍼 좌표계는 관례상 +Y 가 위이므로, 표준 row 는 **아래에서 위로**, 표준 y 는 **die 상단에서 아래로** 증가한다고 읽는 것이 자연스럽다. 다만 이 “위/아래”는 파일에서 직접 확인되지 않으므로 **현물 웨이퍼 맵 대조가 유일한 근거**다.

매칭에는 영향이 없지만, 사람이 읽을 때는 헷갈린다

매칭은 (col,row) 를 이산 키로만, (x,y) 를 die 안에서만 쓰므로 두 방향이 반대여도 결과는 정확하다. 그러나 엑셀의 col 5 / row 3 과 y 31035 μm 를 현물 웨이퍼 맵과 대조할 때는 “row 가 커지는 쪽”과 “y 가 커지는 쪽”이 반대라는 점을 알고 봐야 한다.

§ 7 실측 검증

dev/좌표 확인/ 의 이미지 9장 전부에 대해 저장소의 `coords.resolve()` 를 그대로 실행한 결과다. 아래 값은 손으로 계산한 것이 아니라 파서의 실제 출력이다.

Camtek — INI 파서 실행 결과 (6장)

이미지	X	Y	Col	Row	col	row	x	y
예시1 / 272646.165679...	272647.761	165675.854	7	3	5	3	11913.861	30959.654
예시1 / 272649.165672...	—	—	7	3	5	3	11915.107	30956.962
예시2 / 254256.73925...	254256.503	73925.757	6	1	4	5	30770.303	29020.357
예시3 / 288822.91551...	288823.044	91551.612	7	2	5	4	28089.144	1740.812
예시3 / 288831.91573...	—	—	7	2	5	4	28099.288	1754.905
예시3 / 288834.91563...	—	—	7	2	5	4	28100.288	1752.921

KLA — .001 파서 실행 결과 (3장). native_x/native_y 는 원본 XREL/YREL 보존값

이미지	XREL	YREL	XI	YI	col	row	x	y
예시1 / W6459076XYG1_2_0_23_2	11819.421	13870.779	2	0	5	3	11819	31035
예시2 / W6459081XYE1_1_2_14_2	30688.000	15809.670	1	2	4	5	30688	29096
예시3 / W6460162XYF4_2_1_7_1	28010.781	43081.939	2	1	5	4	28011	1823

같은 결함 3쌍의 정합 결과 — 기본 허용 오차 500 μm

쌍	Camtek (col,row)	KLA (col,row)	col 차	row 차	die 내부 거리	판정
예시 1	(5, 3)	(5, 3)	0	0	121.1 μm	✓ 매치
예시 2	(4, 5)	(4, 5)	0	0	111.8 μm	✓ 매치
예시 3	(5, 4)	(5, 4)	0	0	113.4 μm	✓ 매치

남는 100 μm 대의 차이는 정상이다

(col,row) 는 **정확히** 일치하고, die 내부 거리만 110~121 μm 남는다. 이는 두 장비의 측정 오차 + KLA 쪽 정수 반올림 + DiePitch 미세 불일치 (§ 10 ②)가 합쳐진 값으로, 허용 오차 500 μm 의 1/4 수준이다. 이 여유가 있어야 실제 라인 데이터에서도 견딘다.

이 정합은 한 번 깨진 적이 있다

`CAMTEK_ROW_TOTAL` 이 7 이던 시절에는 위 3쌍 모두 **row** 가 **정확히 1** 어긋났고, 엑셀의 row 표기도 현물 웨이퍼 맵보다 +1 이었다. 6 으로 교정한 뒤 정확 일치한다. 회귀 테스트

`dev/tests/test_coords.py::test_real_pair_kla_camtek_rows_align` 이 이 정합을 고정하고 있다.

§ 8 매칭 엔진 — 좌표를 어떻게 쓰는가

CoordScheduler (QThread)는 이미지 유사도를 전혀 계산하지 않는다. SlotPrecomputeWorker · EfficiencyScheduler 와 동일한 시그널 계약을 따르므로 화면 쪽 연결 코드는 그대로 재사용된다.

8.1 전체 흐름

```
CoordScheduler.run()                                workers/coord_matcher.py:151
├── ① 좌표 일괄 프리패치    resolve_batch(모든 ref/val 경로) → coord_cache
│   └── phase("좌표 파싱 중...")    ※ 폴더별 lru_cache 로 INI/.001 은 한 번만 읽힘
├── ② 조기 종료            ref 가 있는데 좌표를 얻은 ref 가 0개면 → failed(COORD_NO_DATA_MSG)
├── ③ 슬롯별 루프
│   ├── val 을 (col,row) → [(path, x, y), ...] 맵으로 색인
│   ├── ref 별:
│   │   ├── 좌표 있음 → _match_neighbors() → results[(slot, ref)] = [(val, score), ...]
│   │   │                               빈 목록이면 failed_set 에 추가
│   │   └── 좌표 없음 → fallback_refs 에 적립
│   └── fallback_refs → score_ref_classical()    ※ 이미지 유사도로 풀백
```

8.2 (col, row) ±1 이웃 게이트

aoi_verification/app/workers/coord_matcher.py:78

```
tol3 = tol * 3.0 if tol > 0 else 0.0
within3: List[Tuple[Path, float]] = []
for dc in (-1, 0, 1):
    for dr in (-1, 0, 1):
        # 3×3 = 9칸 탐색
        for vpath, vx, vy in val_coord_map.get((ref_col + dc, ref_row + dr), ()):
            dist = math.hypot(ref_x - vx, ref_y - vy) # die 내부 유클리드 거리 (μm)
            if dist <= tol3:
                within3.append((vpath, dist))
return _select_coord_candidates(within3, tol)
```

±1 은 임의의 여유가 아니라 정답 도구의 규약이다

기준이 되는 도구 **AOI Data Viewer** 의 VBA Module_Compare.AOIMapCompare 가 `Abs(col차)<=1 And Abs(row차)<=1` 로 ±1 을 허용한다. 두 장비의 die 인덱스가 1 어긋날 수 있어(실제로 7-Row 시절 전 구간 어긋났다), 정확 일치만 요구하면 매칭이 전멸한다. 현재는 정확 일치하지만 방어용으로 게이트는 유지한다.

8.3 후보 선택과 score 인코딩

aoi_verification/app/workers/coord_matcher.py:53

```

CONFIDENT_DIST = 20.0  # μm

def _select_coord_candidates(within3, tol):
    def score_of(dist: float) -> float:
        if dist <= tol:
            return max(0.0, 1.0 - dist / tol) if tol > 0 else 1.0  # 양수
            return -(dist / tol) if tol > 0 else -1.0              # 음수

    ordered = sorted(within3, key=lambda x: x[1])  # 거리 오름차순
    if not ordered:
        return []
    if ordered[0][1] <= CONFIDENT_DIST:
        ordered = ordered[:1]  # 확신 → 1장만
    return [(p, score_of(d)) for p, d in ordered]

```

거리 조건	score	검토 화면 동작
$\text{dist} = 0$	1.0	완벽 일치
$\text{dist} \leq 20 \mu\text{m}$	≈ 1.0	‘거의 정확히 일치’ → 후보 1장만 노출
$20 \mu\text{m} < \text{dist} \leq \text{tol}$	0.0 ~ 1.0 (양수)	허용 범위 내. $3 \times \text{tol}$ 이내 후보를 전부 차순위로 노출
$\text{tol} < \text{dist} \leq 3 \times \text{tol}$	-3.0 ~ 0.0 (음수)	‘허용범위 초과’ 표식과 함께 노출
$\text{dist} > 3 \times \text{tol}$	—	후보 제외. 남는 후보가 없으면 failed_set

검토 화면은 score 를 거리로 역산해 표시하므로, 이 인코딩은 round-trip 이 보장되어야 한다: $\text{score} \geq 0 \rightarrow \text{dist} = (1 - \text{score}) \times \text{tol}$, $\text{score} < 0 \rightarrow \text{dist} = (-\text{score}) \times \text{tol}$. 인코딩을 바꾸면 match_review_page._format_score 와 _RunnerUpTile 도 함께 고쳐야 한다.

8.4 허용 오차 설정

기본값 500.0 μm — config.py:131 SimilarityConfig.coord_tolerance

UI 슬라이더 10.0 ~ 5000.0 μm — setup_page.py coord_tol_spin

모드 진입 EngineMode.COORDINATE — utils/prefs.py:38

좌표 전무 i18n.KO.COORD_NO_DATA_MSG 로 실패 종료

§ 9 결과 표기와 테스트 맵

9.1 엑셀 출력

미매칭 행의 D열(파일명 아래)에 회색 8pt 로 좌표를 덧붙인다. 매칭 단계와 같은 `coords.resolve()` 를 그대로 재사용하므로 표기와 매칭이 어긋날 수 없다.

aoi_verification/app/workers/exporter.py:517

```
c = resolve_coord(Path(path))
if c is None:
    return []
grey = InlineFont(sz=8, color="FF808080")
blocks = [
    TextBlock(grey, f"\ncol {c.col} / row {c.row}"),
    TextBlock(grey, f"\nx {c.x:.0f} / y {c.y:.0f} μm"),
]
# KLA 결함은 위 변환값(Camtek 좌표계)에 더해 자체 원본 좌표(XREL/YREL)도 표기.
if (c.source == "kla" and c.native_x is not None
    and c.native_y is not None):
    blocks.append(TextBlock(grey, i18n.K0.EXPORT_KLA_NATIVE_FMT.format(
        x=c.native_x, y=c.native_y)))
```

예시 1(KLA)의 실제 출력 모양:

```
W6459076XYG1_2_0_23_2.jpg
col 5 / row 3
x 11819 / y 31035 μm
KLA 원본 x 11819.4 / y 13870.8 μm
```

9.2 화면 표기

좌표 모드에서 `match_review_page._format_score` 는 score 를 μm 로 역산해 "121 μm" 또는 "1500 μm (허용범위 초과)" 로 보여주고, 정렬은 **score 오름차순**(= 거리가 먼 것부터)이라 확인이 필요한 건이 위로 온다. 반면 **매치 실패 검토 다이얼로그**는 **항상 이미지 유사도(%)로 표시**한다 — 좌표 매칭에 실패한 이미지는 좌표를 신뢰할 수 없으므로 시각적 유사도가 더 적절한 기준이라는 의도적 설계다(`unmatched_review_dialog.py` 의 `self._coord_mode = False`).

9.3 테스트 맵

테스트	의존성	검증 내용
test_coords.py	없음 (순수)	INI 변환 5예시 · FaultX 폴백 · 필수키 누락 → None · LIVE 두 형식 · KLA 파일명 오매칭 거부 3건 · 우선순위(LIVE > INI) · KLA 폴스루
test_coords.py:: test_real_pair_...	실측 데이터	회귀 고정 — 예시1 쌍이 양쪽 다 (5,3) 인지. dev/좌표 확인 / 없으면 skip
test_coord_matcher.py	PyQt6	정확 일치·±1 이웃·대각 이웃 매칭 / 차이 2 거부 / 3×tol 초과 실패 / CONFIDENT 1장 규칙 / score 부호·값 / 실예시 end-to-end
test_diagnose_kla_camtek.py	없음 (순수)	진단 도구의 offset/flip 추정 헬퍼
test_kla_wafer_id.py	없음 (순수)	read_wafer_id 의 후보 전수 탐색 · 확장자 없는 정보파일

변환식을 고칠 때의 최소 절차

1. 먼저 `python dev/diagnose_kla_camtek_coords.py <KLA폴더> <Camtek폴더>` 로 systematic offset/flip 을 **실측으로** 드러낸다 (추측 금지 — 오매칭은 정확도 파손이다).
2. 상수/식을 고친다.
3. `QT_QPA_PLATFORM=offscreen python -m pytest -q` 전체 통과 확인.
4. 회귀 테스트의 기대값도 함께 갱신했다면, **왜** 바뀌었는지 커밋 메시지에 남긴다.

§ 10 알려진 한계와 주의사항

① 모든 상수가 TB500 전용 하드코딩

PITCH · OFFSET · ZERO 여섯 값 전부 TB500 기준이다. 다른 제품에 쓰면 col/row/x/y 가 전부 틀리는데 **예외가 나지 않고 그럴듯한 값이 나와 조용히 오매칭된다**. 제품 판별·방어 로직은 없다. 다제품 지원은 models.py 의 상수를 제품별로 분기하는 것에서 시작해야 한다.

② DiePitch 값의 출처가 섞여 있다

축	Camtek 계산에 쓰는 값	KLA 계산에 쓰는 값	차이
X	37247.7 (상수)	쓰지 않음	— (헤더값 37247.93 과 0.23 μm 차)
Y	44905.4 (상수)	44905.34 (파일 헤더)	0.06 μm

KLA 쪽 y 는 파일에서 읽은 DiePitchY 를, Camtek 쪽은 상수를 쓴다. 차이가 die 내부 잔차라 현재 오차($\approx 110 \mu\text{m}$)에 묻히지만, **출처가 통일되어 있지 않다**는 점을 알고 있어야 한다.

③ KLA DefectRecordSpec 을 읽지 않는다

파일에 DefectRecordSpec 22 DEFECTID X Y XREL YREL XINDEX YINDEX ... 로 스키마가 명시돼 있는데, 파서는 정규식으로 위치 0~6 을 고정 가정한다(kla_info.py:31). 레시피·장비 설정에 따라 필드 순서가 달라지면 **예외 없이 엉뚱한 값이 좌표가 된다**.

④ 정보 파일 탐색의 비대칭

좌표용 _find_info_file 은 .001 이 없고 비-사진 파일이 2개 이상이면 None 을 돌려 **그 폴더 전체가 좌표 없음**이 된다(→ 유사도 폴백). 반면 같은 폴더의 read_wafer_id 는 후보를 전부 훑는다. 그래서 **슬롯명은 읽혔는데 좌표는 못 읽는 상태**가 실제로 가능하다.

⑤ LIVE 파일명 경로는 실측 검증이 없다

dev/좌표 확인/ 에는 INI·KLA 샘플만 있고 LIVE 샘플이 없어, test_coords.py 의 LIVE 테스트는 전부 **합성 문자열**이다. 그런데 이 경로는 col 은 그대로 두고 row 만 -1 하는 비대칭 정규화를 하므로, 전제가 틀리면 row 가 통째로 1 어긋난다 — 과거 7-Row 버그와 같은 유형이다. LIVE 실물 샘플을 확보하면 회귀 테스트를 붙일 가치가 크다.

⑥ row 인덱스와 y 의 증가 방향이 반대 (§ 6)

매칭에는 영향이 없다. 다만 엑셀·UI 표기를 현물 웨이퍼 맵과 대조할 때 혼동 소지가 있다.

⑦ LRU 캐시는 파일 변경을 감지하지 않는다

load_folder · load_folder_raw · load_abs_folder · read_wafer_id 모두 maxsize=256 의 폴더 단위 캐시다. **파일이 바뀌어도 다시 읽지 않는다**. 테스트나 재스캔 시에는 cache_clear() 가 필요하다(실제로 회귀 테스트가 그렇게 한다).

§ A 부록 A. 용어집

용어	뜻
die	웨이퍼를 잘라 만드는 칩 하나. 이 문서에서는 웨이퍼 격자의 “칸”.
die pitch	이웃한 die 원점 사이의 거리(= 칸 크기). TB500 은 $37247.7 \times 44905.4 \mu\text{m}$.
col / row	이 앱의 표준 die 인덱스(0-based). Camtek·KLA 양쪽을 이 값으로 통일한다.
Col / Row	Camtek 원본 die 인덱스(INI 의 키). 대문자로 구분해 쓴다.
XINDEX / YINDEX	KLA 원본 die 인덱스. 웨이퍼 중심 die 가 0 인 부호 있는 정수.
XREL / YREL	KLA 의 die 내부 상대 좌표(μm). 변환 후에도 <code>native_x/y</code> 로 보존된다.
tolerance (tol)	die 내부 거리 허용 오차(μm). 기본 500. $3 \times \text{tol}$ 까지는 차순위 후보로 노출.
slot	웨이퍼 한 장(= WaferID). KLA 정보 파일 헤더의 <code>WaferID "..."</code> 가 1순위 출처.
ref / val	기준(reference) 이미지 / 검증(validation) 이미지. 매칭은 ref 마다 val 후보를 찾는다.
LIVE	Camtek 의 저장 모드 중 하나. 파일명 자체에 좌표가 인코딩된다.

§ B 부록 B. 파일·함수 인덱스

좌표 계산에 직접 관여하는 코드 (행 번호는 작성 시점 기준)

위치	역할
coords/__init__.py:20	<code>resolve()</code> — 3파서 우선순위 체인
coords/__init__.py:31	<code>resolve_batch()</code> — 일괄 처리(캐시 활용)
coords/models.py:14	<code>DefectCoord</code> 데이터 클래스
coords/models.py:44	TB500 변환 상수 6개
coords/camtek_ini.py:52	<code>_parse_ini()</code> — 섹션 split 파싱
coords/camtek_ini.py:68	<code>_extract_coord()</code> — Camtek 변환식
coords/camtek_ini.py:110	<code>load_abs_folder()</code> — 절대 좌표(Surface.flt 매칭용)
coords/camtek_live.py:36	<code>_extract()</code> — LIVE 파일명 변환
coords/camtek_live.py:47	<code>resolve()</code> — KLA 파일명 오매칭 방지 검사
coords/kla_info.py:53	<code>_find_info_file()</code> — 정보 파일 탐색
coords/kla_info.py:88	<code>read_wafer_id()</code> — 슬롯명 판독(좌표와 별개 규칙)
coords/kla_info.py:128	<code>load_folder()</code> — KLA 변환식
coords/kla_info.py:141	<code>_parse_info_raw()</code> — TiffFileName/DefectList 상태 기계
coords/abs_coord.py:28	<code>absolute_xy()</code> — 절대 좌표 전용 경로(변환 없음)
workers/coord_matcher.py:53	<code>_select_coord_candidates()</code> — 후보 선택·score 인코딩
workers/coord_matcher.py:78	<code>_match_neighbors()</code> — ±1 이웃 게이트
workers/coord_matcher.py:111	<code>CoordScheduler</code> — 매칭 스케줄러(QThread)
workers/exporter.py:517	<code>_coord_blocks()</code> — 엑셀 좌표 표기
dev/diagnose_kla_camtek_coords.py	두 폴더의 변환 어긋남 실측 진단 도구

이 보고서의 수치는 저장소의 파서를 직접 실행하고 `dev/좌표 확인/` 의 실물 파일을 파싱해 검증했습니다.
 변환식·상수를 수정한 경우 이 문서와 `dev/tests/test_coords.py` 를 함께 갱신하세요.